simcompare 따라하기

더미 데이터로 처음부터 결과 읽기까지

가짜(더미) 데이터를 만들고, 점검·학습·분석·평가를 실제로 돌린 뒤 나온 결과를 그림으로 읽는 과정을 순서대로 보여 줍니다. 이 문서의 모든 숫자와 그림은 아래 명령을 실제로 실행해서 얻은 값입니다.

약 11초	21	6	20개
전체 실행 시간(CPU)	만든 사례 수	비교한 제품 수	만들어진 결과 파일

> **꼭 기억할 점** — 여기서 쓴 데이터는 간단한 수식으로 만든 **가짜**입니다. 그림 속 결과는 “프로그램이 이렇게 동작한다”를 보여 줄 뿐, 실제 제품에 대한 결론이 아닙니다.

차례

1. 이 도구는 무엇을 하나요
2. 전체 흐름 한눈에 보기
3. 1단계 · 더미 데이터 만들기
4. 2단계 · 데이터 점검
5. 3단계 · 모델 학습
6. 4단계 · 사례를 좌표로 바꾸기
7. 5단계 · 분석 (위험 비교 · 순위 · 비슷한 사례 · 검토 카드)
8. 6단계 · 평가
9. 결과를 읽는 순서와 주의할 점
10. 내 데이터로 바꿔 실행하기

1 이 도구는 무엇을 하나요

디스플레이 공정에서는 아주 작은 먼지(**이물**)가 떨어져 전극이 끊어지거나 서로 붙는 불량이 생깁니다. 설계팀은 제품마다 "이물이 떨어지면 얼마나 위험한지"를 이미 시뮬레이션으로 계산해 둡니다. **simcompare**는 이렇게 **이미 계산된 결과**를 여러 제품끼리 나란히 놓고 비교하는 도구입니다.

할 수 있는 것

- 여러 제품에서 반복되는 위험 유형 찾기
- "이 사례와 비슷한 다른 제품 사례" 찾기
- 제품 크기·공장(Fab) 차이를 반영하면 위험 순위가 어떻게 바뀌는지 보기
- 설계자가 검토할 근거를 한 장짜리 **검토 카드**로 정리

하지 않는 것

- 새 설계의 위험(CA·hotspot)을 **예측**하기
- 설계를 자동으로 고치기
- "이것이 원인이다"라고 **확정**하기
- 수율·불량률 계산하기

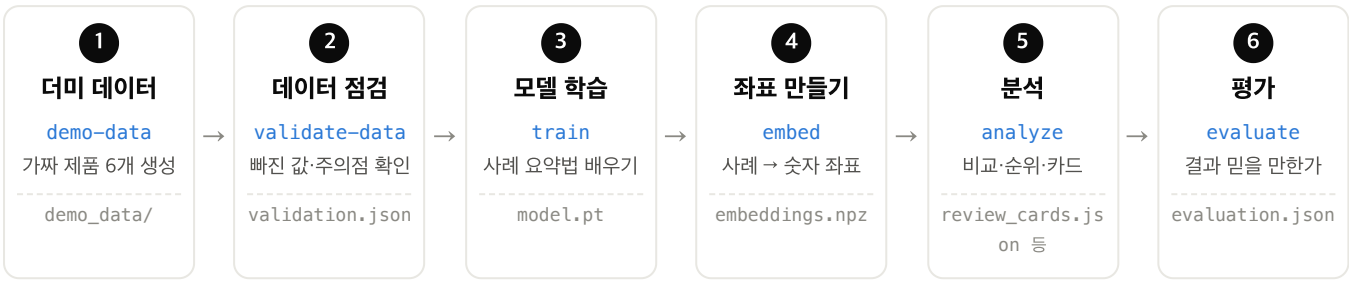
먼저 알아 둘 용어 (9개면 충분합니다)

용어	쉬운 설명
이물	공정 중에 떨어지는 작은 먼지 입자. 크기(μm)별로 다릅니다.
layout (전극 그림)	전극이 어떻게 생겼는지 보여 주는 흑백 그림.
hotspot	전극 그림 위에서 **불량이 잘 나는 자리**를 진하게 표시한 그림. 이물 크기마다 한 장씩.
CA	이물이 떨어졌을 때 불량으로 이어지는 정도를 **0~1 사이 값**으로 나타낸 위험값. 클수록 이물에 약한 설계입니다.
면적당 CA	CA를 화소(unit-pixel) 한 칸의 면적으로 나눈 값. 화소가 큰 제품은 CA도 커지기 쉬워서, 크기가 다른 제품끼리 공정하게 비교할 때 씁니다.
Fab 분포	실제 공장(Fab)에서 관측한 **이물 크기별 비율**. 공장마다 다릅니다.
CAA	CA에 Fab 분포를 곱해 더한 값. "이 공장 기준으로 얼마나 취약한가"를 보는 **비교용 지표**이며 수율이 아닙니다.
사례	제품 × 설계 버전 × 불량 종류 한 묶음. 예) `P1-v1-OPEN` = 제품 P1, 버전 v1, 끊김 불량.
요약 좌표	사례 하나를 숫자 몇 개로 줄인 것. 좌표가 가까우면 비슷한 사례로 봅니다.

불량 종류는 세 가지입니다: **끊김**(lateral_open, 전극 1개), **옆 전극 붙음**(lateral_short, 같은 층 전극 2개), **위아래 전극 붙음**(vertical_short, 다른 층 전극 2개).

2 전체 흐름 한눈에 보기

여섯 단계를 순서대로 실행합니다. 각 단계는 앞 단계가 만든 파일을 읽어서 다음 파일을 만듭니다.



준비 (처음 한 번만)

Python 3.11 이상과 `uv`가 필요합니다. GPU나 인터넷 연결 없이 일반 노트북 CPU로 충분합니다.

```
$ uv sync
```

```
$ uv run pytest        # 모든 테스트가 통과하면 설치 완료
```

여섯 단계를 한 번에 실행하기

```
$ uv run simcompare all --config configs/demo.yaml --demo-out demo_data
```

결과는 `outputs/demo/`에 저장됩니다. 실행 중에 `DEMO 모드: 합성/예시 설정이며 결과는 연구 근거가 아닙니다` 라는 문구가 보이면 정상입니다. 이 문서를 만들 때는 전체가 **약 11초** 걸렸습니다.

아래부터는 같은 과정을 **한 단계씩** 실행하며 무엇이 만들어지는지 봅니다.

단계	명령	걸린 시간
1	`simcompare demo-data`	0.1초
2	`simcompare validate-data`	1.6초
3	`simcompare train`	2.9초
4	`simcompare embed`	1.4초
5	`simcompare analyze`	2.0초
6	`simcompare evaluate`	2.9초

설정 파일 `configs/demo.yaml`의 주요 값: 이물 크기 1·2·4·8·16 μm, 학습 20회 반복, 평가용 비율 25%, seed 0. seed가 같으면 몇 번을 돌려도 같은 결과가 나옵니다.

3 1단계 · 더미 데이터 만들기

```
$ uv run simcompare demo-data --out demo_data --seed 0
```

실제 자료가 없어도 전체 흐름을 시험할 수 있도록 **가짜 제품 6개**(P1~P6)를 만듭니다. 제품마다 불량 3종류가 있고, P1은 설계를 한 번 바꾼 v2가 더 있어 **사례는 모두 21개**입니다.

```
demo_data/
├── bundles.csv        사례 목록 (제품·버전·불량 종류·공장·크기 등)
├── layouts.csv        전극 그림 파일 위치
├── hotspots.csv       hotspot 그림 파일 위치 (이물 크기별)
├── ca_records.csv     이물 크기별 CA 값
├── fab.json           공장별 이물 크기 분포
├── geometry.json      두께·전극 간격·선폭 같은 형상 수치
├── arrays/, images/   실제 그림 파일
└── summary.json       생성 요약
```

사례 하나는 이렇게 생겼습니다



그림 1. P1 v1의 '옆 전극 붙음' 사례. 왼쪽 두 장은 전극 그림, 오른쪽 세 장은 이물 크기별 hotspot입니다. 이물이 클수록 위험한 자리가 넓고 진해집니다.

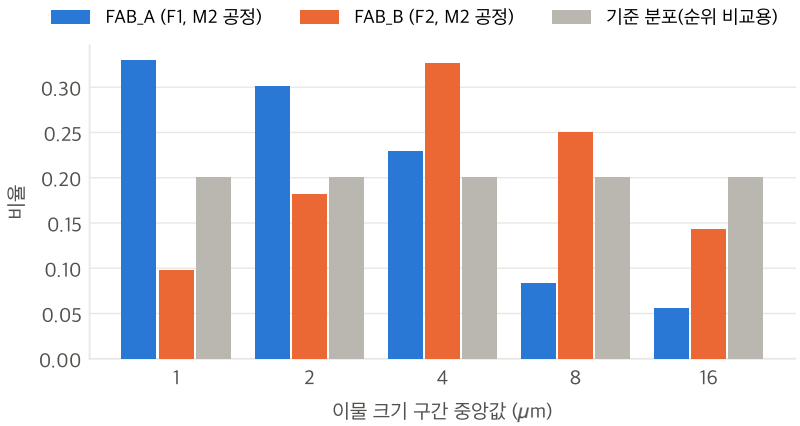


그림 2. 공장별 이물 크기 분포. FAB_A는 작은 이물이, FAB_B는 중간~큰 이물이 많습니다. 회색은 순위 비교용으로 모든 크기를 똑같이 둔 기준 분포입니다.

4 2단계 · 데이터 점검

```
$ uv run simcompare validate-data --config configs/demo.yaml
```

학습 전에 자료 상태를 확인합니다. 자료를 고치지는 않고 **보고만** 합니다. 화면에는 요약이, 파일 `validation.json`에는 자세한 내용이 남습니다.

```
{
  "n_bundles": 21,
  "n_warnings": 7,
  "confounding": "ok"
}
```

1단계에서 일부러 넣어 둔 '빈틈'

실제 자료처럼 값이 빠지거나 조건이 다른 경우를 섞어 두었습니다. 이 단계가 이를 제대로 알려 주는지 확인합니다.

- **P6**은 실제 크기(μm) 정보가 없습니다 → 면적 기준 비교에서 빠져야 합니다.
- **P4**는 8 μm hotspot과 형상 수치가 없습니다.
- **P3**는 CA 2 μm 값이, **P2**는 CA 16 μm 값이 없습니다.
- **P1 v2**는 P1 v1에서 전극 간격을 넓힌 변경 설계입니다.
- **P5**는 다른 제품보다 위험 수치를 3배 크게 만들었습니다.

	전극 그림	hotspot 2	hotspot 4	hotspot 8	CA 1	CA 2	CA 4	CA 8	CA 16	실제 크기	형상 수치
P1	있음	있음	있음	있음	있음	있음	있음	있음	있음	있음	있음
P2	있음	있음	있음	있음	있음	있음	있음	있음	**없음**	있음	있음
P3	있음	있음	있음	있음	있음	**없음**	있음	있음	있음	있음	있음
P4	있음	있음	있음	**없음**	있음	있음	있음	있음	있음	있음	**없음**
P5	있음	있음	있음	있음	있음	있음	있음	있음	있음	있음	있음
P6	있음	있음	있음	있음	있음	있음	있음	있음	있음	**없음**	있음

그림 3. 제품별로 어떤 자료가 있는지 정리한 표 (validation.json과 입력 파일에서 계산). 주황 칸이 빠진 자료입니다 — 1단계에서 넣어 둔 빈틈이 모두 잡혔습니다.

이 결과를 어떻게 읽나요

확인 항목	이번 결과	의미
실제 크기 없음	P6-v1-LSHORT, P6-v1-OPEN, P6-v1-VSHORT	이 사례는 면적당 CA 비교에서 빠집니다.
빠진 CA 값	2 μm 3건, 16 μm 3건	빈 값을 0으로 채우지 않고 '없음'으로 둡니다.
hotspot ≈ 전극 그림	5건 (P2, P3, P4, P5, P6의 끊김)	hotspot이 전극 모양을 거의 그대로 따라가 같은 정보를 두 번 쓸 수 있다는 주의.
공장 × 제품군 겹침	ok	제품군 F3가 두 공장에 모두 있어 공장 효과와 제품군 효과를 **어느 정도** 나눠 볼 수 있습니다.
픽셀 위치 대응 확인 불가	P6-v1-LSHORT, P6-v1-OPEN, P6-v1-VSHORT	전극 그림과 hotspot의 픽셀 크기 정보가 없어 두 그림이 같은 위치를 가리키는지 확인할 수 없습니다.
CA 값 범위	최대 0.034 (0.03 초과 1건)	일반 범위보다 큰 값이 있어 원자료 확인이 필요하다는 안내입니다(오류는 아님).
학습/평가 나누기	가능	서로 독립인 제품 묶음이 6개 있습니다.

경고가 있어도 다음 단계는 실행됩니다. 하지만 실제 자료라면 **경고부터 읽고** 자료를 보완할지 먼저 판단하세요.

5 3단계 · 모델 학습

```
$ uv run simcompare train --config configs/demo.yaml
```

전극 그림·hotspot·CA·Fab 분포·형상 수치를 한꺼번에 받아 **짧은 숫자 묶음으로 압축했다가 다시 원래대로 복원**하도록 모델(오토인코더)을 연습시킵니다. 복원을 잘하려면 사례의 핵심을 담아야 하므로, 압축된 숫자가 곧 그 사례의 **요약**이 됩니다.

연습용과 시험용을 제품 단위로 나눕니다

학습용 · 15개 사례

P1 P2 P5 P6

평가용 · 6개 사례

P3 P4

같은 제품이나 같은 설계 계열(예: P1 v1과 v2)은 **반드시 한쪽에만** 들어갑니다. 연습문제와 시험문제가 겹치면 실력을 부풀려 보게 되기 때문입니다. 나눈 결과는 `split.json`에 있습니다.

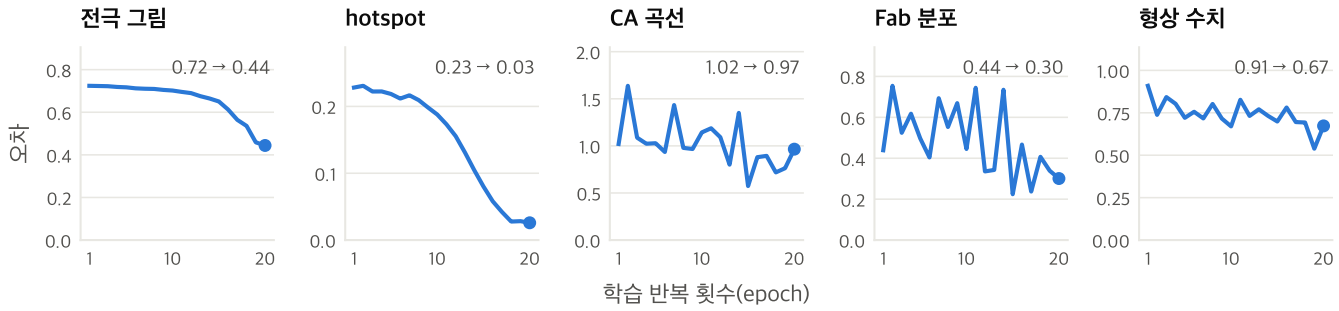


그림 4. 학습 반복에 따른 입력별 복원 오차 (train_history.csv). 낮을수록 잘 복원한다는 뜻입니다. 칸마다 세로축 범위가 다릅니다.

이 결과를 어떻게 읽나요

- **hotspot**(0.23 → 0.03)과 **전극 그림**(0.72 → 0.44)은 꾸준히 줄었습니다.
- **CA·Fab·형상 수치**는 오르내림이 큽니다. 학습 사례가 15개뿐이라 자연스러운 모습입니다.
- 오차가 작다고 해서 비교·검색 결과가 좋다는 보장은 **없습니다**. 그 판단은 5·6단계에서 합니다.

함께 만들어지는 파일: `model.pt`(학습된 모델), `train_history.csv`(그림 4의 원자료), `split.json`(나눈 결과), `run_record.json`(seed·설정·버전 기록).

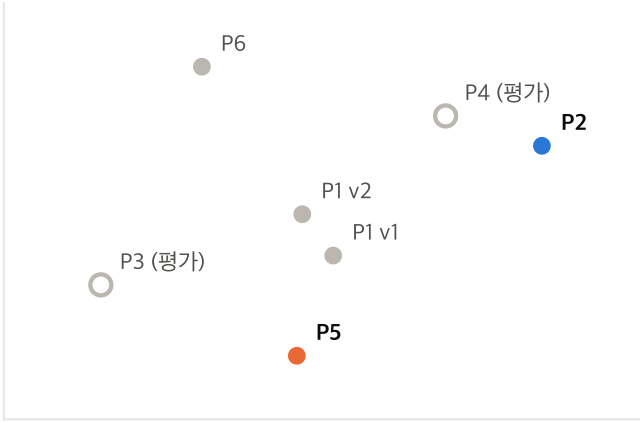
6 4단계 · 사례를 좌표로 바꾸기

```
$ uv run simcompare embed --config configs/demo.yaml
```

모든 사례를 숫자 좌표로 바꿉니다. 좌표를 만드는 방법이 다섯 가지이고, 각각을 '공간'이라고 부릅니다. 가장 중요한 구분은 **위험 수치(CA·hotspot)를 썼느냐**입니다.

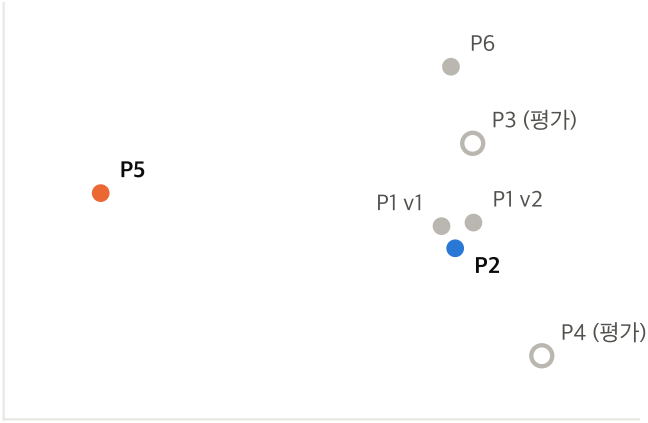
공간 이름	무엇으로 만들었나	위험 수치 사용	크기
`shape_only`	전극 모양·실제 크기·두께 같은 **모양 정보만** (단순 통계 방법)	아니오	8개 숫자
`with_risk_baseline`	모양 + hotspot·CA·Fab 요약값 (단순 통계 방법)	예	8개 숫자
`with_risk_ae`	3단계에서 학습한 모델의 요약	예	32개 숫자
`ae_struct`	모델 요약 중 설계·시뮬레이션 쪽 부분	예	16개 숫자
`ae_fab`	모델 요약 중 공장 분포 쪽 부분	예	16개 숫자

① 모양 정보만 사용 (shape_only)



가로·세로축은 요약 좌표 (단위·의미 없음, 거리만 봄)

② 위험 수치까지 사용 (with_risk_ae)



가로·세로축은 요약 좌표 (단위·의미 없음, 거리만 봄)

● 채운 점 = 학습에 쓴 제품　　○ 속 빈 점 = 평가용 제품

그림 5. 끊김 사례 7개를 두 공간에서 평면 지도로 그린 것 (embeddings.npz를 2차원으로 줄여 표시). 점 사이 거리가 가까울수록 비슷한 사례입니다.

이 결과를 어떻게 읽나요

- ① 모양만 보면 **P5**는 다른 제품들 사이에 섞여 있습니다.
- ② 위험 수치를 넣으면 **P5가 홀로 멀리** 떨어집니다. P5의 CA가 다른 제품보다 약 7배 크기 때문입니다.
- **P1 v1과 v2**는 두 지도 모두에서 가깝습니다. 설계를 조금 바꾼 같은 계열이기 때문입니다.

> **주의** — 위험 수치를 넣어 만든 공간에서 "위험이 비슷한 사례"를 찾은 것은 새로운 발견이 아닙니다. 입력한 값을 다시 본 것에 가깝습니다. 결과 파일에서는 `reuses_risk_inputs=True` 로 표시됩니다.

7 5단계 · 분석

```
$ uv run simcompare analyze --config configs/demo.yaml
```

분석 단계는 네 가지 질문에 답하는 파일을 만듭니다. 이번 실행에서 검토 카드 3장, 비슷한 사례 315줄이 만들어졌습니다.

질문	보는 파일
제품별로 위험이 얼마나 다른가?	`risk_table.csv`
크기·공장을 반영하면 순위가 바뀌나?	`ranking.csv`, `rank_agreement.json`
이 사례와 비슷한 다른 제품 사례는?	`neighbors.csv`, `clusters.json`
결과가 제품 하나에 좌우되지 않나?	`loo_stability.csv`
한 장으로 정리하면?	`review_cards.json`
그림으로 한눈에 보려면?	`figures/ca_curves.svg`, `figures/loo_stability.svg`, `figures/rank_shift.svg`

7-1. 제품별 위험 비교

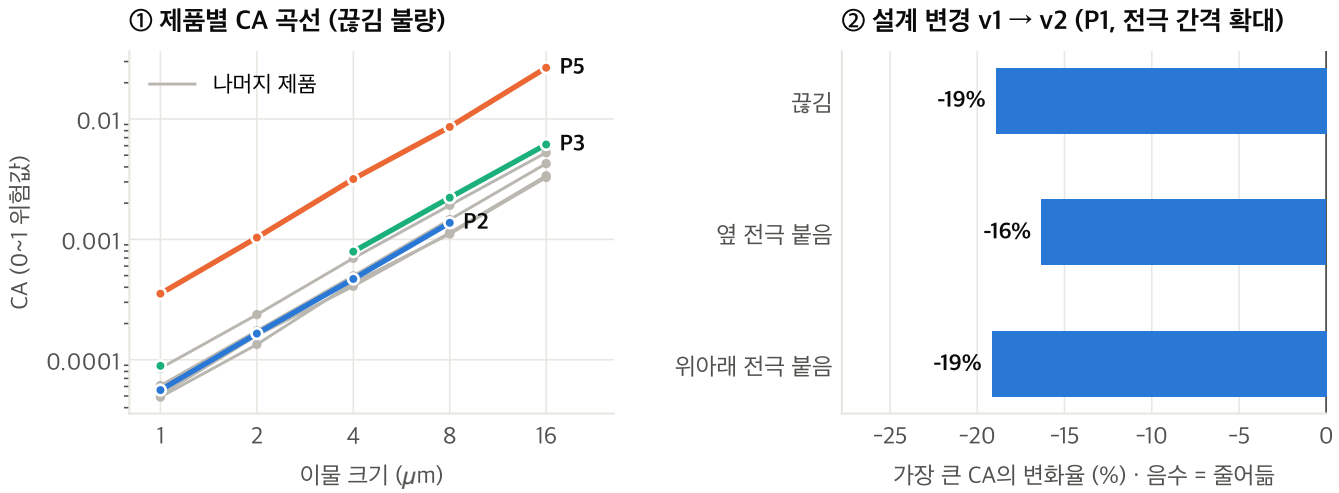
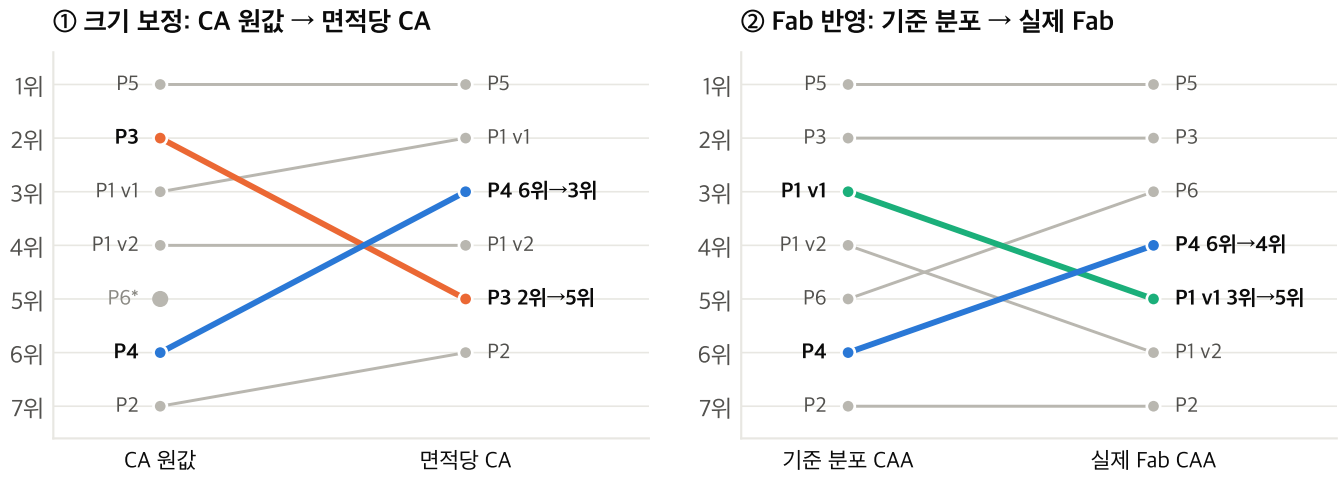


그림 6. ① 끊김 불량의 이물 크기별 CA (risk_table.csv, 양쪽 축 모두 로그 눈금). ② P1 설계 변경 전후 가장 큰 CA의 변화율.

- ① **P5**는 모든 이물 크기에서 CA가 가장 큽니다. **P3**는 2 μm 값이, **P2**는 16 μm 값이 빠져 선이 끊겨 있습니다 — 없는 값을 지어내지 않습니다.
- ② 전극 간격을 넓힌 **P1 v2**는 세 불량 모두 가장 큰 CA가 16~19% 줄었습니다. 다만 이것은 가짜 데이터의 수식이 그렇게 만든 결과입니다.

7-2. 크기·공장을 반영하면 순위가 바뀝니다



* P6: 실제 크기 정보가 없어 면적당 CA를 계산하지 않음

그림 7. 끊김 불량 7개 사례의 위험 순위 변화 (ranking.csv). 1위가 가장 위험합니다. 크게 바뀐 사례만 색으로 표시했습니다.

- ① **P3**는 CA 원값으로는 2위지만 면적당으로 보면 5위입니다. 화소가 커서(80×120 μm) CA가 크게 나온 것입니다. 반대로 작은 제품 **P4**는 6위 → 3위로 올라갑니다.
- ② **P4**는 실제 공장(FAB_B) 분포를 쓰면 6위 → 4위로 올라갑니다. FAB_B에 큰 이물이 많기 때문입니다. 작은 이물이 많은 FAB_A의 **P1 v1**은 3위 → 5위로 내려갑니다.
- 두 순위가 얼마나 비슷한지는 `rank_agreement.json`에 0~1 사이 값(1이면 완전히 같음)으로 나옵니다: 끊김 기준 공장 반영 전후 0.71, 크기 보정 전후 0.60.

> CAA는 이번 실행에서 **DEMO 규칙**으로 계산됐습니다. 실제 자료에서는 계산 규칙이 확정되기 전까지 CAA 칸이 이유와 함께 비어 있습니다.

7-3. 비슷한 사례 찾기

`neighbors.csv`에는 사례마다 공간별로 가장 가까운 다른 제품 사례 3개가 있습니다. 같은 불량 종류·같은 전극 역할끼리만 비교합니다. 아래는 가장 위험한 **P5 끊김** 사례의 결과입니다.

공간	가까운 순서	위험 수치 사용
`shape_only` 모양만	P1 v1 → P1 v2 → P6	**아니오**
`with_risk_baseline` 모양+위험 (통계)	P1 v1 → P2 → P1 v2	예 (독립 발견 아님)
`with_risk_ae` 모양+위험 (모델)	P1 v1 → P6 → P2	예 (독립 발견 아님)
`ae_struct` 모델 · 설계 쪽	P3 → P1 v1 → P6	예 (독립 발견 아님)
`ae_fab` 모델 · 공장 쪽	P6 → P1 v1 → P1 v2	예 (독립 발견 아님)

공간마다 답이 조금씩 다릅니다. **모양만 쓴 공간**(P1 v1 → P1 v2 → P6)이 "위험과 상관없이 모양이 비슷한 사례"라는 독립적인 근거이고, 나머지는 참고용입니다.

7-4. 제품 하나를 빼도 결과가 유지되나요

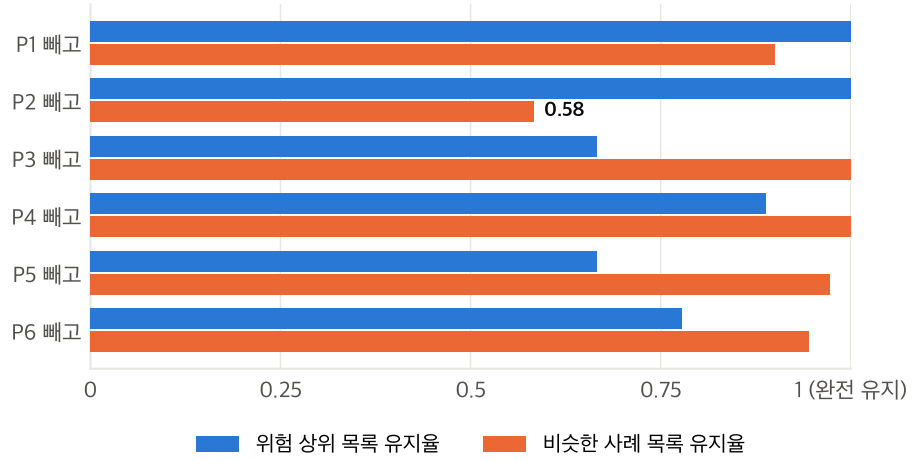


그림 8. 제품을 하나씩 빼고 다시 분석했을 때 목록이 얼마나 그대로인지 (loo_stability.csv). 1이면 완전히 같고 0이면 전혀 겹치지 않습니다.

- **P2**를 빼면 비슷한 사례 목록이 0.58만 유지됩니다. 여러 사례의 '이웃' 역할을 P2가 많이 하고 있다는 뜻입니다.
- **P3**처럼 위험 상위 제품을 빼면 상위 목록이 0.67로 떨어집니다. 상위 목록이 바뀌는 건 당연하므로 크게 걱정할 일은 아닙니다.
- 값이 낮은 제품이 있으면 "그 제품 하나 때문에 나온 결론인지" 확인해 보세요.

7-5. 검토 카드 (review_cards.json)

불량 종류·전극 역할·layer 조합마다 한 장씩, 앞의 결과를 모아 정리합니다. 아래는 끊김 불량 카드를 그대로 옮긴 것입니다.

card-1 **끊김 불량 · 전극 역할 data · layer M2** 비교 가능 · 제품 6개

대표 사례
P5
이 그룹에서 위험이 높은 쪽을 대표

반대 사례
P2, P1 v2
같은 그룹이지만 위험이 낮은 쪽

위험 높은 사례들의 공통 형상
M2 두께, 전극이 차지하는 비율, 전극 간격
값이 서로 비슷했던 특징 (원인이라는 뜻 아님)

비교할 수 없는 조건
unit-pixel 실제 크기 없음(면적 보정 비교 불가): ['P6-v1-OPEN']

설계 변경 검토 후보 (improvement_candidates)

형상	위험 높은 사례(중앙값)	위험 낮은 사례(중앙값)	검토해 볼 방향
전극 경계 밀도	0.0227	0.0237	늘리는 쪽
전극 간격	9	6	줄이는 쪽

전문가 확인 필요

- 제품 간 전극 역할·layer 대응 확인
- CAA 가중 규칙과 Fab 자료 단위 확인
- 누락된 이물 크기의 CA 재계산 필요: ['P2-v1-OPEN', 'P3-v1-OPEN']
- CAA 값은 DEMO 규칙으로 계산됨: 해석 전에 확정된 규칙으로 교체 필요

검증 상태
이물 재시뮬레이션 대기 1건 · 외부 전기/설계 검토 필요 6건

검토 후보는 "위험이 높은 사례와 낮은 사례에서 값이 달랐던 형상"일 뿐입니다. 바꾸면 좋아진다는 뜻이 아니며, 이물 재시뮬레이션과 전기 특성(저항·정전용량 등) 검토를 거쳐야 합니다.

7-6. 자동으로 저장되는 그림 (figures/)

`analyze`를 실행하면 위에서 본 표 가운데 세 가지가 그림으로도 저장됩니다. 파일은 `outputs/demo/figures/`에 있고, **브라우저로 열면** 한글 설명까지 그대로 보입니다. 추가 설치는 필요 없습니다.

파일	보여 주는 것	이 문서의 비슷한 그림
`rank_shift.svg`	크기 보정·Fab 반영 전후 순위 변화. 가장 크게 바뀐 두 사례만 색으로 표시	그림 7
`ca_curves.svg`	이물 크기별 CA 곡선. CA가 가장 큰 사례를 주황으로 표시하고 빠진 값은 선을 끊음	그림 6 ①
`loo_stability.svg`	제품을 하나씩 뺐을 때 두 목록의 유지율 막대	그림 8

이 문서의 그림은 설명을 위해 일부 사례만 골라 그렸지만, 자동 그림은 **모든 비교 그룹**(불량 종류·전극 역할·layer 조합)을 한 줄씩 세로로 쌓아 보여 줍니다. 아래는 이번 실행에서 실제로 저장된 `rank_shift.svg`의 첫 번째 그룹입니다.

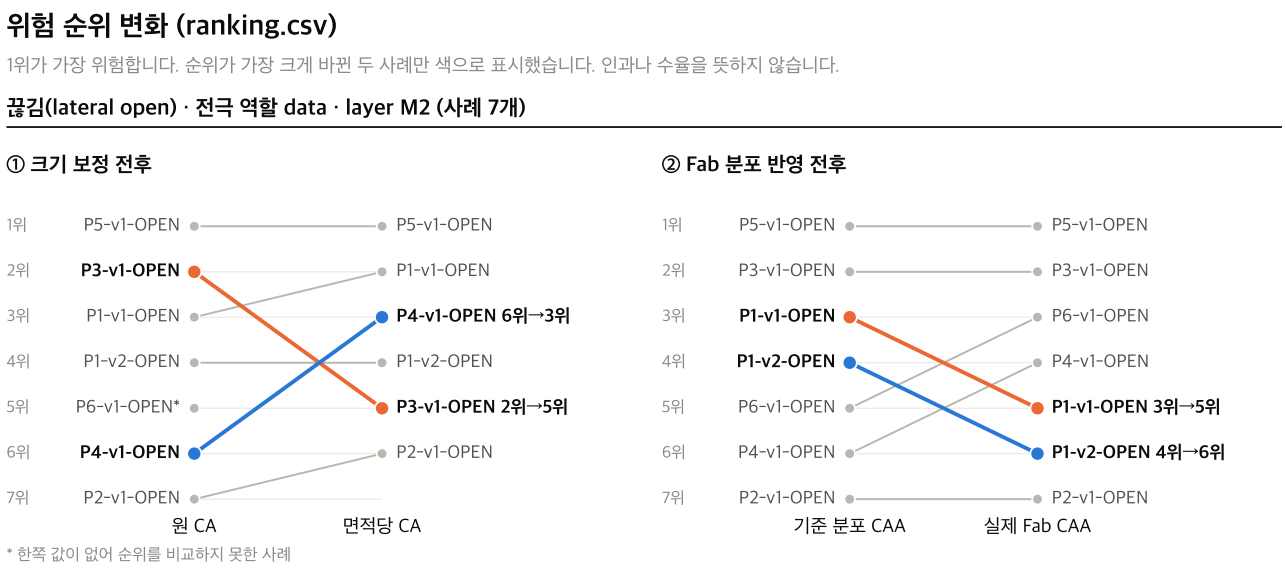


그림 10. analyze가 자동으로 저장한 figures/rank_shift.svg의 윗부분 (끊김 불량 그룹). 이름 뒤 *는 한쪽 값이 없어 순위를 비교하지 못한 사례입니다.

> 그림은 결과 표를 옮긴 것일 뿐 새로운 판단을 더하지 않습니다. 사례 ID가 길면 그림에서는 줄여 표시하고, 전체 ID는 CSV 파일에 있습니다.

8 6단계 · 평가

```
$ uv run simcompare evaluate --config configs/demo.yaml
```

학습에 쓰지 않은 평가용 제품(P3, P4)으로 결과를 점검합니다. 결과는 `evaluation.json`에 있고, 학습·평가가 섞이지 않았는지 검사(`leakage_check`)는 **passed**였습니다.

8-1. 입력을 섞으면 도움이 되나요

같은 조건으로 '그림만', '숫자만', '모두' 넣은 모델을 각각 학습해 평가용 제품을 얼마나 잘 복원하는지 비교합니다.

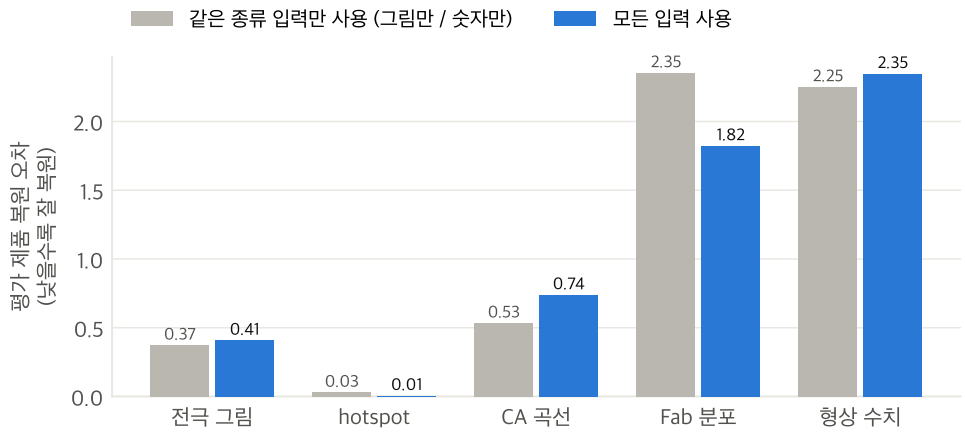


그림 9. 입력 종류별 평가 복원 오차 (evaluation.json의 ablation). 회색은 같은 종류 입력만 넣은 모델, 파랑은 전부 넣은 모델입니다.

- 모두 넣었을 때 오차가 줄어든 입력: **hotspot·Fab** / 오히려 늘어난 입력: **전극 그림·CA·형상 수치**.
- 평가 제품이 2개뿐이라 이 차이로 "섞는 게 낫다/못하다"를 판단할 수 **없습니다**. 실제 자료에서 다시 확인해야 합니다.
- 각 모델이 찾은 비슷한 사례가 '모양만 쓴 공간'과 얼마나 겹치는지(0~1): 그림만 0.44, 숫자만 0.53, 모두 0.44.

8-2. 처음 보는 제품에서도 경향이 반복되나요

평가용 사례마다 **학습용 제품 중 모양이 가장 비슷한 사례**를 찾습니다. 그다음 평가 사례의 면적당 CA와 이웃들의 평균이 **둘 다 학습 사례의 중간값보다 높거나, 둘 다 낮으면** '같음'으로 셉니다. 정답표가 없으므로 정확도가 아니라 "모양이 비슷하면 위험 경향도 비슷한가"를 보는 참고 근거입니다.

평가 사례	모양이 비슷한 학습 사례	위험 경향
`P3-v1-OPEN`	P1 v2 · P1 v1 · P6	다름
`P3-v1-LSHORT`	P1 v2 · P1 v1 · P6	다름
`P3-v1-VSHORT`	P1 v2 · P1 v1 · P6	다름
`P4-v1-OPEN`	P2 · P1 v1 · P1 v2	같음
`P4-v1-LSHORT`	P2 · P1 v1 · P1 v2	같음
`P4-v1-VSHORT`	P2 · P1 v1 · P1 v2	다름

경향이 같았던 비율: **0.33** (6건 중). 평가 제품이 2개뿐이라 이 값 하나로 판단하지 않습니다.

8-3. 공장·제품군을 통째로 빼 보기

빼 본 기준	판정	설명
공장(Fab)	주의해서 평가 가능 (ok)	공장 하나를 통째로 빼고 나머지로 설명이 되는지 봅니다.
제품군	주의해서 평가 가능 (ok)	제품군 하나를 통째로 빼고 봅니다.

판정은 세 가지 중 하나입니다. **주의해서 평가 가능**: 비교할 수 있음. **평가 불가**: 뺄 그룹이 부족함. **효과 분리 불가**: 공장과 제품군이 늘 같이 움직여 둘 중 무엇의 영향인지 구분할 수 없음.

> 복원 오차가 작다는 것은 “요약을 잘했다”는 뜻일 뿐, “비교·해석이 옳다”는 뜻이 아닙니다.

9 결과를 읽는 순서와 주의할 점

결과 파일이 20개나 되지만, 아래 순서로 보면 충분합니다.

1. **`validation.json`의 경고** — 빠진 자료, 실제 크기 없는 제품, 공장·제품군 겹침부터 확인합니다. (그림 3)
2. **`split.json`** — 어떤 제품이 평가용으로 갔는지 확인합니다. (5장)
3. **`review_cards.json`** — 불량 종류별 카드에서 대표·반대 사례, 검토 후보, '자료 없음', '전문가 확인 필요'를 읽습니다. (7-4)
4. **`ranking.csv`** — 크기·공장 반영 전후로 순위가 크게 바뀐 제품을 찾습니다. `figures/rank_shift.svg`로 먼저 훑어보면 빠릅니다. (그림 7·10)
5. **`neighbors.csv`** — 모양만 쓴 공간(`reuses_risk_inputs=False`)의 이웃을 우선 봅니다. (7-3)
6. **`loo_stability.csv`**, **`evaluation.json`** — 결론이 제품 하나에 좌우되지 않는지 확인합니다. (그림 8·9)

헷갈리기 쉬운 세 가지

비슷하다 ≠ 원인이다
이 도구는 "같이 나타난다"까지만 보여줍니다. 원인 판단은 설계 전문가의 몫입니다.

CAA ≠ 수율
CAA는 제품끼리 비교하기 위한 지표입니다. 실제 불량률·수율로 읽으면 안 됩니다.

군집 번호에는 뜻이 없음
`clusters.json`의 0번·1번은 이름표일 뿐, 0번이 더 안전하다는 뜻이 아닙니다.

10 내 데이터로 바꿔 실행하기

1. **자료 정리**: 1단계의 `demo_data/`와 같은 모양으로 `bundles.csv`, `fab.json` 등을 준비합니다. 자세한 열 이름은 `docs/data_contract.md`에 있습니다.
2. **설정 복사**: `configs/real.example.yaml`을 `configs/real.yaml`로 복사하고 자료 위치와 이물 크기 목록을 바꿉니다.
3. **점검 먼저**: `uv run simcompare validate-data --config configs/real.yaml` 후 경고를 읽습니다.
4. **나머지 실행**: train → embed → analyze → evaluate 순서로 같은 설정 파일을 넣어 실행합니다.

> 실제 모드(`mode: real`)에서는 확정되지 않은 계산을 하지 않습니다. 예를 들어 CAA 규칙이 정해지기 전에는 CAA 칸이 비어 있고, 위아래 전극 붙음 사례에 적용 공정 layer가 없으면 오류로 멈춥니다. 확정이 필요한 항목은 `docs/open_questions.md`에 있습니다.

자주 만나는 메시지

화면에 나온 문구	해결
`…model.pt이(가) 없습니다. 먼저 'simcompare train'을 실행하세요`	train을 먼저 실행합니다.
`체크포인트의 샘플이 현재 데이터에 없습니다(재학습 필요)`	자료가 바뀌었으니 train부터 다시 실행합니다.
`알 수 없는 설정 키 …`	설정 파일 오타입니다. `configs/real.example.yaml`과 비교합니다.
`독립 그룹(제품·설계 계열)이 2개 미만입니다`	비교할 제품이 너무 적습니다. 제품을 늘립니다.

더 자세한 설명: `docs/usage.md`(실행 가이드), `docs/model_design.md`(모델 구조와 한계). 이 문서는 `uv run --with matplotlib python docs/walkthrough/build.py`로 다시 만들 수 있습니다.